

DECENT: 온라인 MEC Offloading의 지연·자원 결정

Deep Reinforcement Learning for Online Latency Aware Workload
Offloading in Mobile Edge Computing

Zeinab Akhavan, Mona Esmaeili, Babak Badnava, Mohammad Yousefi, Xiang Sun, Michael Devetsikiotis, Payman Zarkesh-Ha (2022)

대기열과 task priority를 관측해 MEC 목적지와 compute resource를 함께 고르는 A2C-style 접근

가장 가까운 MEC server가 항상 최선은 아닌 이유

network delay가 작아도 compute resource와 두 대기열의 상태가 나쁘면 response time은 커질 수 있다

01	task 도착	BS에 이미 offload할 task가 도착
↓		
02	가까운 server	전송과 E2E delay는 작을 수 있음
↓		
03	server 상태	resource 부족과 queue 적체 가능
↓		
04	응답 지연	network와 compute delay를 함께 합산
↓		
05	공동 결정	목적지와 resource 양을 같이 선택

논문의 핵심은 거리 하나가 아니라 queue·resource·priority를 함께 보는 것이다

이 논문이 결정하는 것과 결정하지 않는 것

task는 이미 offloading 대상으로 정해져 있으며, BS가 server와 compute resource를 선택한다

범위	논문이 다루는 결정	범위 밖
offloading	어느 MEC server로 보낼지	local execution 여부
resource	선택한 server의 discrete resource level	연속 resource control
지연	network와 computing queue wait 포함	결과 반환 지연
목표	weighted response time 최소화	energy를 목적식에 추가

‘offload할까’가 아니라 ‘이미 offload할 task를 어디에, 얼마의 resource로 보낼까’의 문제다

MEC 시스템에서 BS가 보는 두 종류의 대기열

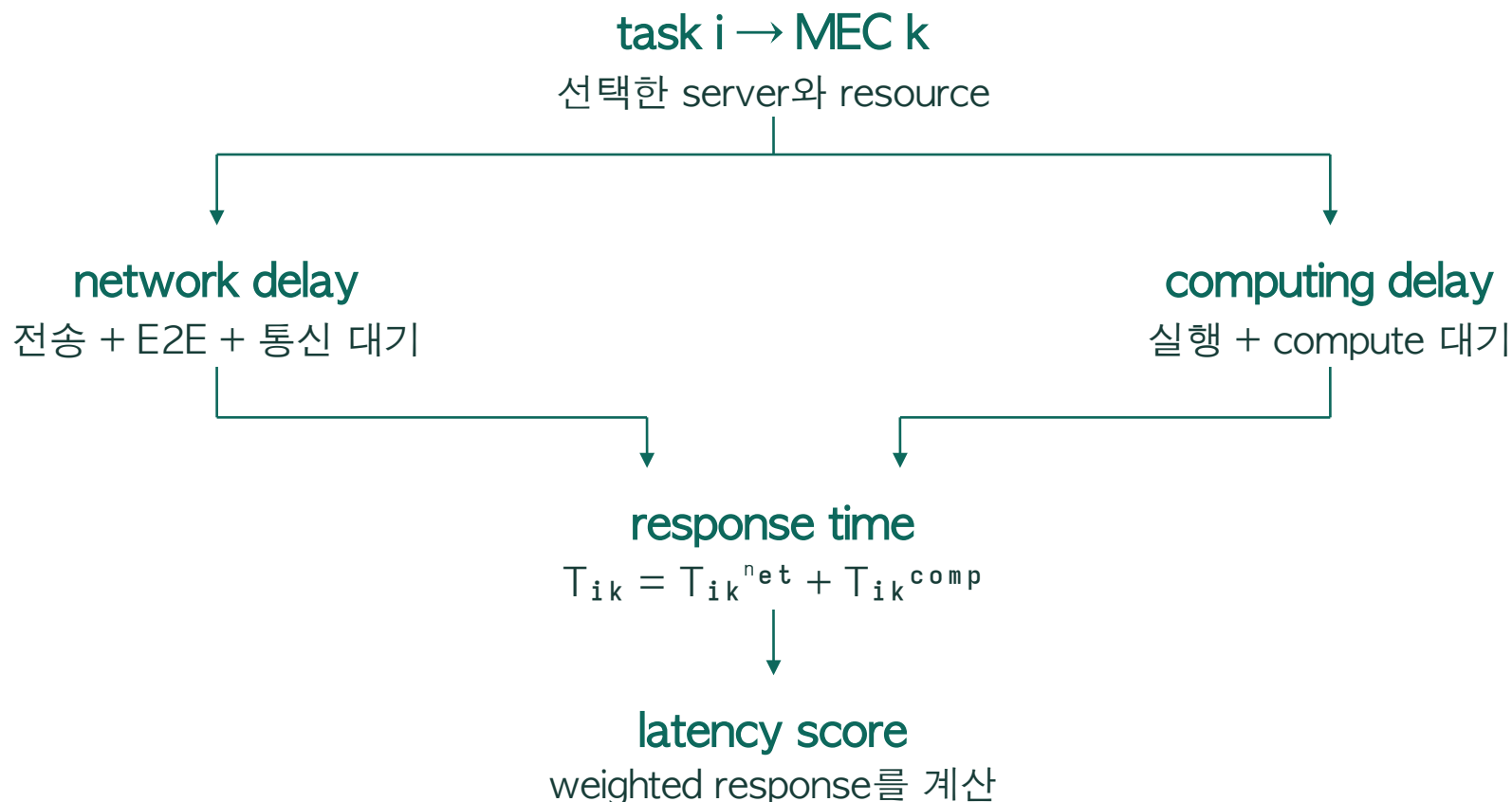
BS의 destination별 communication queue와 MEC server의 computing queue가 서로 다른 지연을 만든다



communication queue와 computing queue를 함께 모델링하는 것이 DECENT의 문제 정의다

response time은 network path와 computing path를 합친다

목적지 선택과 resource 선택은 각기 다른 항목을 바꾸지만 최종 지연에서는 결합된다



P0: priority를 반영한 response time 최소화

각 task는 정확히 한 MEC server를 고르고, resource는 허용된 discrete block 중 하나여야 한다

요소	정의	의미
목적	$\sum_i \eta_i \sum_k x_{ik} (T_{ik}^{comp} + T_{ik}^{net})$ 최소	weighted response time
server	$\sum_k x_{ik} = 1$	task마다 목적지 하나
indicator	$x_{ik} \in \{0, 1\}$	server 선택 여부
resource	$r_{ik} \in \{0, R_1, \dots, R^{max}\}$	허용된 discrete resource

priority η 가 클수록 같은 response time이 objective에 더 크게 반영된다

도착 순간의 MDP state는 무엇인가

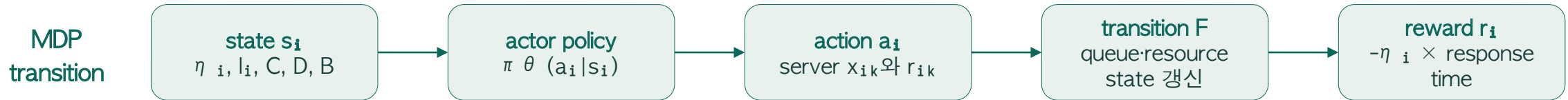
state는 새 task 정보 하나와 모든 MEC server의 resource-queue 상태를 함께 담는다

state item	원문 정의	결정에 주는 정보
η_i, l_i	새 task의 priority와 data size	어떤 지연을 더 중시할지
$C = \{c_k\}$	각 server의 remaining compute resource	실행 가능한 resource 여유
$D = \{d_k\}$	각 computing queue의 workload	compute 대기량
$B = \{b_k\}$	각 communication queue의 wait	전송 전 대기량

DECENT는 server의 거리만 아니라 현재 남은 resource와 양쪽 queue state를 입력으로 쓴다

action-reward: server와 resource의 짝

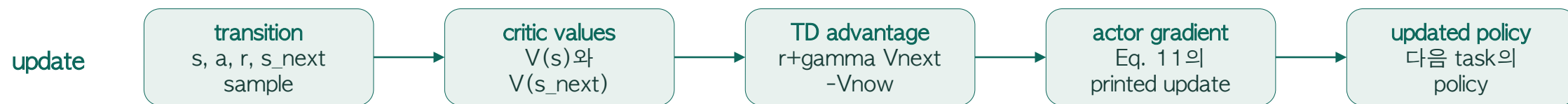
actor는 task i 마다 $a_i = \{r_{ik}, x_{ik}\}$ 를 고르고, 환경은 다음 state와 reward를 만든다



negative reward를 크게 만든다는 것은 weighted response time을 작게 만든다는 뜻이다

A2C에서 actor와 critic이 나누는 일

replay에서 꺼낸 transition의 TD advantage가 원문 Eq. 11 actor update에 들어간다



advantage는 critic value와 reward를 actor update에 연결하는 학습 신호다

원문의 actor-critic update와 구현 경계

논문은 update 식을 제시하지만 actor objective의 부호와 action encoding의 구현 세부를 정하지 않는다

원문 항목	명시된 내용	강의에서 지킬 경계
actor update	$\theta \leftarrow \theta - \beta_a \nabla_{\theta} J(\theta)$	J 최대화 설명과 sign convention 구분
critic update	TD residual 제곱을 $J(v)$ 로 최소화	Eq. (12)–(13) 범위만 설명
action output	softmax action distribution	pair encoding·mask· ϵ 연계 미명시
network	actor-critic hidden layer 각 128 ReLU	추가 layer·target network 주장 안 함

원문의 수식은 제시하되, 미명시 code-level 선택을 논문 주장으로 만들지 않는다

Algorithm 1 : training에서 transition을 모아 update한다

논문은 task-arrival transition collection 뒤 replay sampling과 actor-critic update를 수행한다고 적는다

01	초기화	$\gamma, \beta_a, \beta_c, \epsilon$ 설정
↓		
02	task arrival	state 관측 후 ϵ -greedy action
↓		
03	reward	r_i 계산, next state 획득
↓		
04	replay buffer	(s, a, r, s') transition 저장
↓		
05	update	sample 후 actor와 critic 갱신

10% random·90% greedy action은 원문이 명시한 exploration setting이다

강의용 trace: reward와 advantage를 직접 계산해 보기

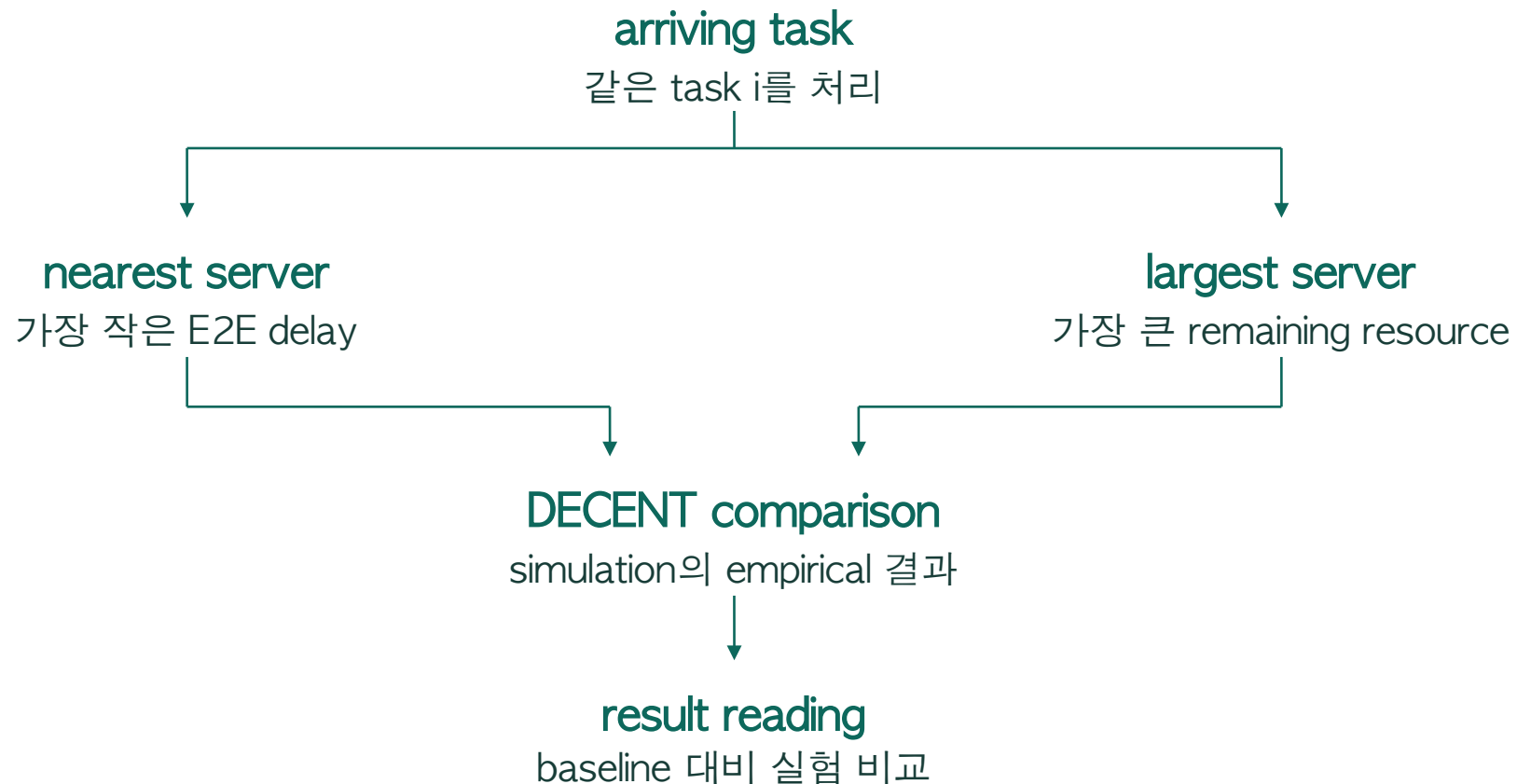
아래 candidate delay와 critic value는 수식 연습을 위해 만든 값이며 논문 결과가 아니다

항목	MEC 1 action A	MEC 2 action B
action	server 1, resource 100	server 2, resource 60
가정 response time	0.12 s	0.20 s
reward, $\eta = 20$	$-20 \times 0.12 = -2.4$	$-20 \times 0.20 = -4.0$
advantage	$-2.4 + 0.9(-1) - (-4) = +0.7$	$-4.0 + 0.9(-1) - (-4) = -0.9$

이 invented critic baseline에서는 A가 B보다 더 좋은 one-step advantage를 보인다

두 baseline은 서로 다른 한 가지 신호만 본다

DECENT의 비교는 nearest server와 largest remaining-resource server라는 두 휴리스틱에 한정된다



simulation 조건 1 : topology와 resource

모든 결과는 one BS와 four MEC server, 명시된 E2E-capacity-discrete resource set에서 얻었다

항목	보고된 조건
topology	1 BS; 4 MEC server; distance 0, 1, 2, 3 km
E2E delay	$T = \alpha d + \zeta$; initially $\alpha = .03$ s/km, $\zeta = .03$ s
server capacity	$R_{max} = 200$ Mcycles/s
link capacity	$w = 2 \times 10^9$ bits/s
resource r	$\{10, 20, 40, 60, 80, 100, 120, 140, 160, 200\}$ Mcycles/s

논문의 성능 주장은 이 simulation model과 두 baseline의 비교 범위에서 읽어야 한다

simulation 조건 2: workload와 training

task size의 normal-distribution 두 번째 parameter는 원문 표기 그대로 두며 의미를 추정하지 않는다

항목	보고된 조건
arrival	Poisson $\lambda = 50$ tasks/s
task size	$l_i \sim N(3 \times 10^7 \text{ bits}, 3 \times 10^5)$
computation	$\mu = .15$ CPU cycles/bit
priority	$\eta_i \sim U\{10, 20, 50, 100\}$
training	$\beta_a = .0001; \beta_c = .0002; \varepsilon = .1$

normal N의 두 번째 parameter는 source가 variance-SD로 명시하지 않는다

무엇이 측정되었고, 어디까지 말할 수 있는가

그림들은 학습과 여러 sensitivity setting의 simulation 결과이며 보편적 guarantee가 아니다

보고된 관찰	조건	말할 수 없는 확장
learning curve	약 500 episodes 부근에서 수렴 관찰	convergence theorem
Fig. 3 response	text: weighted $\langle 1$ s; axis: ms	다른 arrival pattern
arrival sweep	6,400 tasks에서 Fig. 4, 500 ms 미만	모든 load·topology
priority behavior	higher weight가 낮은 execution time	hard deadline 보장
$\mu \cdot a$ sweep	두 baseline보다 lowest average	모든 policy보다 최적

DECENT의 우위는 보고된 simulation 조건에서의 측정 결과이지 일반 정리가 아니다

priority와 execution time: 보고된 관계

큰 η 는 같은 delay에 더 큰 penalty를 주며, Figure 5는 높은 weight task의 낮은 execution time을 보고한다

01 높은 η 같은 T에서 $r=-\eta$ T가 더 음수



02 actor action server와 resource pair 선택



03 resource effect 실행 time과 queue state에 영향



04 Fig. 5 higher weight의 lower execution time 보고

weight는 논문의 priority signal이지, 만족이 보장되는 deadline constraint가 아니다

이 논문을 적용 주장으로 바꾸기 전에 남는 경계

모델의 state 입력과 별도 E2E delay 모델, simulation 구조를 구분해야 한다

경계	source가 다룬 범위	남는 질문
observation	state: η_i, l_i, C, D, B ; E2E는 별도 모델 항	stale·partial telemetry
decision	one server와 discrete resource	local execution·resource encoding
learning	A2C-style update와 replay sampling	sign·encoding 미명시
evidence	one-BS/four-server simulations	production overhead·robustness
guarantee	sub-optimal DRL solution 목표	optimality·convergence theorem

DECENT를 이해한다는 것은 policy의 입력·결정·측정 범위를 함께 이해하는 것이다

DECENT의 한 task decision을 다시 연결하면

목적지와 resource를 함께 고르는 이유는 priority와 양쪽 queue state가 response time을 함께 바꾸기 때문이다

01	관측	η , size, C, D, B를 읽음
↓		
02	선택	server와 discrete resource pair
↓		
03	평가	음의 weighted response reward
↓		
04	학습	replay transition과 actor-critic update
↓		
05	읽기	simulation result와 guarantee 분리

DECENT는 관측된 queue-resource state에 따라 server-resource pair를 고른다